Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 5-9

Study of Morphological Operations on Image

Course Code: ECE 4224

Course Title: Digital Image Processing Sessional

Submitted to:
Oishi Jyoti
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: March 31, 2026

Contents

1 Experiment 5: Image Thresholding **1**
1.1 Theory and Introduction . . . 1
1.2 Methodology . . . 1
1.2.1 Matrix Definition . . . 1
1.2.2 Python Implementation . . . 1
1.3 Results . . . 2
1.4 Discussion . . . 2
1.5 Conclusion . . . 2

2 Experiment 6: Linear Contrast Stretching **3**
2.1 Theory and Introduction . . . 3
2.2 Methodology . . . 3
2.2.1 Parameter Definition . . . 3
2.2.2 Python Implementation . . . 3
2.3 Results . . . 4
2.4 Discussion . . . 4
2.5 Conclusion . . . 4

3 Experiment 7: Morphological Erosion and Dilation **5**
3.1 Theory and Introduction . . . 5
3.2 Methodology . . . 5
3.2.1 Matrix Definition . . . 5
3.2.2 Python Implementation . . . 5
3.3 Results . . . 6
3.4 Discussion . . . 7
3.5 Conclusion . . . 7

4 Experiment 8: Morphological Opening and Closing **8**
4.1 Theory and Introduction . . . 8
4.2 Methodology . . . 8
4.2.1 Matrix Definition . . . 8
4.2.2 Python Implementation . . . 8
4.3 Results . . . 9
4.4 Discussion . . . 10
4.5 Conclusion . . . 10

5 Experiment 9: Hit-or-Miss Transform **11**
5.1 Theory and Introduction . . . 11
5.2 Methodology . . . 11
5.2.1 Image Preparation and ROI Selection . . . 11
5.2.2 Python Implementation . . . 11
5.3 Results . . . 12
5.4 Discussion . . . 13
5.5 Conclusion . . . 13

References **13**

1 Experiment 5: Image Thresholding

1.1 Theory and Introduction

Image thresholding is a fundamental method of image segmentation. It is used to convert a grayscale image into a binary image (black and white) by separating objects of interest from the background based on their pixel intensity [1]. A threshold value T is selected, and each pixel is evaluated against this value:

$$g(x, y) = \begin{cases} 1, & \text{if } f(x, y) \geq T \\ 0, & \text{if } f(x, y) < T \end{cases}$$

This experiment explores the manual implementation of this thresholding logic without relying on built-in binarization functions.

1.2 Methodology

A grayscale matrix was evaluated, and conditional logic was applied pixel-by-pixel to generate the binary output.

1.2.1 Matrix Definition

A 5×5 matrix with an increasing gradient of intensity values (ranging from 10 to 255) was defined to clearly observe the boundary created by the threshold.

$$I = \begin{bmatrix} 10 & 50 & 100 & 150 & 200 \\ 30 & 80 & 120 & 180 & 220 \\ 60 & 110 & 140 & 210 & 250 \\ 90 & 130 & 170 & 230 & 240 \\ 120 & 160 & 190 & 245 & 255 \end{bmatrix}$$

A threshold value of $T = 127$ was selected.

1.2.2 Python Implementation

The implementation avoids built-in thresholding functions. An empty array of zeros was created, and nested loops were utilized to iterate through every coordinate. The intensity value was checked against T, and the corresponding output pixel was set to 1 (or 255 for visualization) or 0.

```python
import numpy as np
import os
import matplotlib.pyplot as plt

# 1. Create a 5x5 grayscale matrix image (values 0-255)
img_exp5 = np.array(
    [
        [10, 50, 100, 150, 200],
        [30, 80, 120, 180, 220],
        [60, 110, 140, 210, 250],
        [90, 130, 170, 230, 240],
        [120, 160, 190, 245, 255],
    ],
    dtype=np.uint8,
)

T = 127  # Threshold value

# 2. Manual thresholding using nested loops
bw_img = np.zeros_like(img_exp5)
rows, cols = img_exp5.shape

for i in range(rows):
    for j in range(cols):
        if img_exp5[i, j] >= T:
            bw_img[i, j] = 1  # Or 255 for display purposes
        else:
            bw_img[i, j] = 0

print("--- Experiment 5: Thresholding ---")
print("Original Matrix:\n", img_exp5)
print(f"B&W Matrix (T={T}):\n", bw_img)
```

```python
# Save the image
output_dir = "./images/output"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, "lab5.png")
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(img_exp5, cmap="gray")
plt.title("Original Image")
plt.subplot(1, 2, 2)
plt.imshow(bw_img, cmap="gray")
plt.title(f"Thresholded (T={T})")
plt.savefig(output_path)
print(f"Image saved to {output_path}")
```

1.3 Results

The code generated a segmented binary matrix, visualized in the figure below.

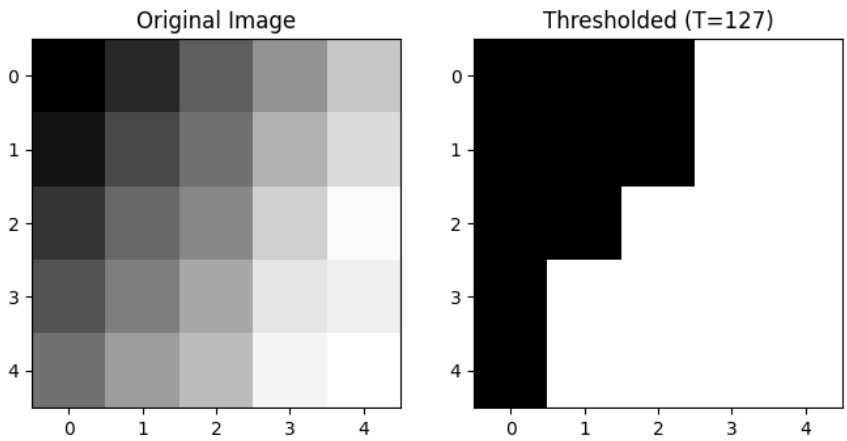

Figure 1: Image Thresholding ($T = 127$). Left: Original grayscale gradient. Right: The resulting binary image showing a clear step-edge segmentation.

1.4 Discussion

The manual conditional mapping successfully segmented the matrix. As observed in the results, all pixels with an intensity ≥ 127 (located towards the bottom right) were assigned the foreground value (white), while those < 127 (top left) were assigned the background value (black). This created a clear diagonal boundary, demonstrating how thresholding groups similar intensity regions.

1.5 Conclusion

The underlying mathematical principle of binary segmentation was validated. By manually applying the threshold piecewise function via iterative loops, it was confirmed that object isolation can be achieved strictly through simple intensity condition checks.

2 Experiment 6: Linear Contrast Stretching

2.1 Theory and Introduction

Linear contrast stretching is a spatial domain enhancement technique used to expand the dynamic range of an image [1]. It employs a piecewise linear transformation function defined by critical points (r_1, s_1) and (r_2, s_2). The transformation is divided into three regions—dark, mid-tone, and bright—each governed by a distinct slope (α, β, γ):

- **Dark region** $(0 \to r_1)$**:** Slope $\alpha = \frac{s_1}{r_1}$
- **Mid-tone region** $(r_1 \to r_2)$**:** Slope $\beta = \frac{s_2 - s_1}{r_2 - r_1}$
- **Bright region** $(r_2 \to L - 1)$**:** Slope $\gamma = \frac{(L-1) - s_2}{(L-1) - r_2}$

2.2 Methodology

The piecewise slopes were calculated, and the transformation was mapped across the pixels of a standard test image (Lenna) to enhance contrast.

2.2.1 Parameter Definition

The input parameters were defined as $r_1 = 70, r_2 = 180$ and target values $s_1 = 40, s_2 = 220$. The maximum intensity level was set to $L - 1 = 255$.

2.2.2 Python Implementation

The slopes α, β, and γ were computed mathematically. Nested loops were used to evaluate each pixel and apply the corresponding linear equation based on which intensity region the pixel occupied.

```python
import numpy as np
import matplotlib.pyplot as plt
import os
from PIL import Image

# 1. Given points
r1, r2 = 70, 180
s1, s2 = 40, 220
L_max = 255

# 2. Calculate slopes (Alpha, Beta, Gamma)
alpha = s1 / r1
beta = (s2 - s1) / (r2 - r1)
gamma = (L_max - s2) / (L_max - r2)

print("\n--- Experiment 6: Contrast Stretching ---")
print(f"Alpha (Dark): {alpha:.2f}")
print(f"Beta (Mid): {beta:.2f}")
print(f"Gamma (Bright): {gamma:.2f}")

# Load Lenna image
img_exp6 = np.array(Image.open("./images/Lenna.png").convert("L"), dtype=np.uint8)
out_exp6 = np.zeros_like(img_exp6, dtype=np.float32)

# 3. Apply piecewise stretching
for i in range(img_exp6.shape[0]):
    for j in range(img_exp6.shape[1]):
        r = img_exp6[i, j]
        if r < r1:
            out_exp6[i, j] = alpha * r
        elif r1 <= r <= r2:
            out_exp6[i, j] = beta * (r - r1) + s1
        else:
            out_exp6[i, j] = gamma * (r - r2) + s2

print("Original min/max:", img_exp6.min(), img_exp6.max())
print("Stretched min/max:", out_exp6.min(), out_exp6.max())

# Save the image
output_dir = "./images/output"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, "lab6.png")

plt.figure(figsize=(12, 5))
```

```python
plt.subplot(1, 2, 1)
plt.imshow(img_exp6, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(np.round(out_exp6).astype(np.uint8), cmap="gray")
plt.title("Stretched Image")
plt.axis("off")
plt.tight_layout()
plt.savefig(output_path, dpi=150, bbox_inches="tight")
print(f"Image saved to {output_path}")
plt.show()
```

2.3 Results

The transformation stretched the mid-tones while compressing extreme shadows and highlights.

Figure 2: Contrast Stretching Results. Left: Original low-contrast image. Right: Stretched image demonstrating enhanced definition and deeper contrast.

2.4 Discussion

The calculated slopes dictate the visual outcome. A slope of $\beta > 1$ in the mid-tone region ($70 \rightarrow 180$) significantly increased the contrast of the subject's features, stretching a narrow band of input values across a wider output range ($40 \rightarrow 220$). Conversely, the dark and bright regions were compressed ($\alpha < 1, \gamma < 1$), which pushed near-black and near-white pixels closer to the absolute limits without clipping. The visual result is a much crisper image.

2.5 Conclusion

The mechanism of piecewise contrast stretching was successfully implemented. It was demonstrated that by mathematically adjusting the linear slopes mapping input to output intensities, specific tonal ranges can be selectively enhanced or suppressed to improve global image quality.

3 Experiment 7: Morphological Erosion and Dilation

3.1 Theory and Introduction

Morphological image processing relies on non-linear operations related to the shape or morphology of features [1]. The two most fundamental operations are:

- **Erosion:** Shrinks or thins objects in a binary image. It is mathematically equivalent to taking the minimum value within the neighborhood defined by a Structuring Element (SE).
- **Dilation:** Grows or thickens objects in a binary image. It is mathematically equivalent to taking the maximum value within the neighborhood defined by the SE.

3.2 Methodology

Morphological operations were manually applied to a synthetic matrix using sliding window arrays rather than built-in morphological functions.

3.2.1 Matrix Definition

A 7×7 binary image matrix with a centered 5×5 square of 1s was defined. A 3×3 Structuring Element (SE) consisting of all 1s was utilized. Zero-padding was applied to the image borders.

3.2.2 Python Implementation

The image was padded with 0s to accommodate the edges. A nested loop structure was used to extract 3×3 windows. For erosion, `np.min(window)` was applied; for dilation, `np.max(window)` was applied.

```python
import numpy as np
import matplotlib.pyplot as plt
import os
from PIL import Image
from scipy.ndimage import binary_dilation, binary_erosion

# 1. Define Image and Structuring Element (SE)
img_exp7 = np.array(
    [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    ],
    dtype=np.uint8,
)

SE = np.ones((3, 3), dtype=np.uint8)


# 2. Manual Erosion (min of window)
def manual_erode(img, se):
    pad = se.shape[0] // 2
    # Padding with 0 as requested in the prompt
    padded = np.pad(img, pad, mode="constant", constant_values=0)
    out = np.zeros_like(img)

    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            window = padded[i : i + se.shape[0], j : j + se.shape[1]]
            out[i, j] = np.min(window)
    return out


# 3. Manual Dilation (max of window)
def manual_dilate(img, se):
    pad = se.shape[0] // 2
    padded = np.pad(img, pad, mode="constant", constant_values=0)
    out = np.zeros_like(img)

    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            window = padded[i : i + se.shape[0], j : j + se.shape[1]]
            out[i, j] = np.max(window)
    return out
```

```python


# Using scipy imdilate equivalent
# dilated_img_scipy = binary_dilation(img_exp7, structure=SE).astype(np.uint8)
# eroded_img_scipy = binary_erosion(img_exp7, structure=SE).astype(np.uint8)


eroded_img = manual_erode(img_exp7, SE)
dilated_img = manual_dilate(img_exp7, SE)

print("\n--- Experiment 7: Morphological Operations ---")
print("Original 7x7:\n", img_exp7)
print("Dilated (Expands 1s):\n", dilated_img)
print("Eroded (Shrinks 1s):\n", eroded_img)
# print("Eroded (Shrinks 1s):\n", eroded_img_scipy)
# print("Dilated (Expands 1s):\n", dilated_img_scipy)

# Save the image
output_dir = "./images/output"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, "lab7.png")

plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(img_exp7, cmap="gray")
plt.title("Original Image")
# Add gridlines
ax1 = plt.gca()
ax1.set_xticks(np.arange(-0.5, img_exp7.shape[1], 1), minor=True)
ax1.set_yticks(np.arange(-0.5, img_exp7.shape[0], 1), minor=True)
ax1.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.subplot(1, 3, 2)
plt.imshow(dilated_img, cmap="gray", vmin=0, vmax=1)
plt.title("Dilated Image")
# Add gridlines
ax2 = plt.gca()
ax2.set_xticks(np.arange(-0.5, dilated_img.shape[1], 1), minor=True)
ax2.set_yticks(np.arange(-0.5, dilated_img.shape[0], 1), minor=True)
ax2.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.subplot(1, 3, 3)
plt.imshow(np.round(eroded_img).astype(np.uint8), cmap="gray")
plt.title("Eroded Image")
# Add gridlines
ax3 = plt.gca()
ax3.set_xticks(np.arange(-0.5, eroded_img.shape[1], 1), minor=True)
ax3.set_yticks(np.arange(-0.5, eroded_img.shape[0], 1), minor=True)
ax3.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.tight_layout()
plt.savefig(output_path, dpi=150, bbox_inches="tight")
print(f"Image saved to {output_path}")
plt.show()
```

3.3 Results

The fundamental shape modifications were recorded in the output matrices.

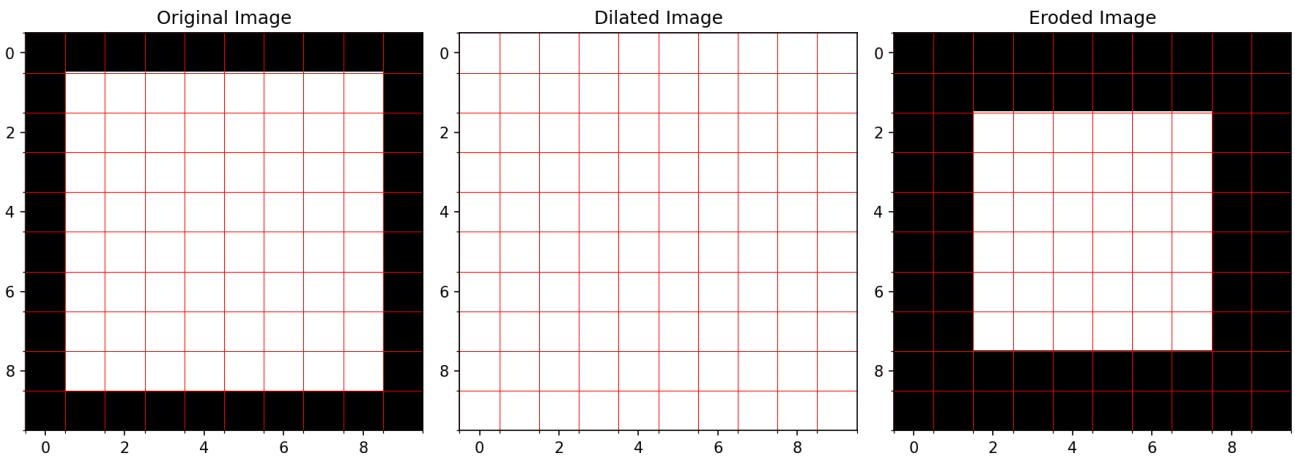

Figure 3: Morphological Basics. The central white shape is reduced via erosion and expanded via dilation (subject to padding constraints).

3.4 Discussion

The sliding window technique directly mirrors morphological theory. During erosion, if even a single zero from the background entered the 3×3 window, the minimum value became 0, effectively stripping the outermost layer of 1s from the square shape. Conversely, during dilation, the presence of a single 1 inside the window forced the center pixel to 1, causing the shape boundaries to expand outward.

3.5 Conclusion

The theoretical concepts of morphological erosion and dilation were verified through manual implementation. It was established that non-linear min and max filters combined with a structuring element effectively manipulate the boundaries of binary objects.

4 Experiment 8: Morphological Opening and Closing

4.1 Theory and Introduction

Opening and closing are secondary morphological operations constructed by combining erosion and dilation in specific sequences [1].

- **Opening ($A \circ B$):** Erosion followed by dilation. It smooths the contour of an object, breaks narrow isthmuses, and eliminates thin protrusions (noise).
- **Closing ($A \bullet B$):** Dilation followed by erosion. It smooths sections of contours, fuses narrow breaks and long thin gulfs, and fills small holes in the object.

4.2 Methodology

A binary image containing geometric artifacts (narrow bridges and gaps) was defined to test both operations using the previously developed erosion and dilation functions.

4.2.1 Matrix Definition

A customized binary matrix was structured to exhibit two large rectangular blocks connected by a narrow horizontal bridge, demonstrating vulnerabilities susceptible to opening and closing.

4.2.2 Python Implementation

The logic was executed sequentially. The `manual_open()` function routed the output of `manual_erode()` directly into `manual_dilate()`. The `manual_close()` function inverted this sequence.

```python
import numpy as np
import matplotlib.pyplot as plt
import os
from PIL import Image

# Define the structuring element
SE = np.ones((3, 3), dtype=np.uint8)


# 2. Manual Erosion (min of window)
def manual_erode(img, se):
    pad = se.shape[0] // 2
    # Padding with 0 as requested in the prompt
    padded = np.pad(img, pad, mode="constant", constant_values=0)
    out = np.zeros_like(img)

    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            window = padded[i : i + se.shape[0], j : j + se.shape[1]]
            out[i, j] = np.min(window)
    return out


# 3. Manual Dilation (max of window)
def manual_dilate(img, se):
    pad = se.shape[0] // 2
    padded = np.pad(img, pad, mode="constant", constant_values=0)
    out = np.zeros_like(img)

    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            window = padded[i : i + se.shape[0], j : j + se.shape[1]]
            out[i, j] = np.max(window)
    return out


# 1. Define Opening and Closing functions using Exp 7 definitions
def manual_open(img, se):
    # Opening = Erode -> Dilate
    return manual_dilate(manual_erode(img, se), se)


def manual_close(img, se):
    # Closing = Dilate -> Erode
    return manual_erode(manual_dilate(img, se), se)


# We use an image with noise to show the effect
img_exp8 = np.array(
```

```python
    [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 1, 1, 1, 0, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    ],
    dtype=np.uint8,
)


opened_img = manual_open(img_exp8, SE)
closed_img = manual_close(img_exp8, SE)

print("\n--- Experiment 8: Opening & Closing ---")
print("Original Image (with sticking out noise):\n", img_exp8)
print("Opened Image (Removes noise):\n", opened_img)
print("Closed Image (Fills holes):\n", closed_img)
# Save the image
output_dir = "./images/output"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, "lab8.png")

plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(img_exp8, cmap="gray")
plt.title("Original Image")
# Add gridlines
ax1 = plt.gca()
ax1.set_xticks(np.arange(-0.5, img_exp8.shape[1], 1), minor=True)
ax1.set_yticks(np.arange(-0.5, img_exp8.shape[0], 1), minor=True)
ax1.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.subplot(1, 3, 2)
plt.imshow(opened_img, cmap="gray")
plt.title("Opened Image")
# Add gridlines
ax2 = plt.gca()
ax2.set_xticks(np.arange(-0.5, opened_img.shape[1], 1), minor=True)
ax2.set_yticks(np.arange(-0.5, opened_img.shape[0], 1), minor=True)
ax2.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.subplot(1, 3, 3)
plt.imshow(closed_img, cmap="gray")
plt.title("Closed Image")
# Add gridlines
ax3 = plt.gca()
ax3.set_xticks(np.arange(-0.5, closed_img.shape[1], 1), minor=True)
ax3.set_yticks(np.arange(-0.5, closed_img.shape[0], 1), minor=True)
ax3.grid(which="minor", color="red", linestyle="-", linewidth=0.5)
plt.tight_layout()
plt.savefig(output_path, dpi=150, bbox_inches="tight")
print(f"Image saved to {output_path}")
plt.show()
```

4.3 Results

The geometric structures were fundamentally altered based on the operation sequence.

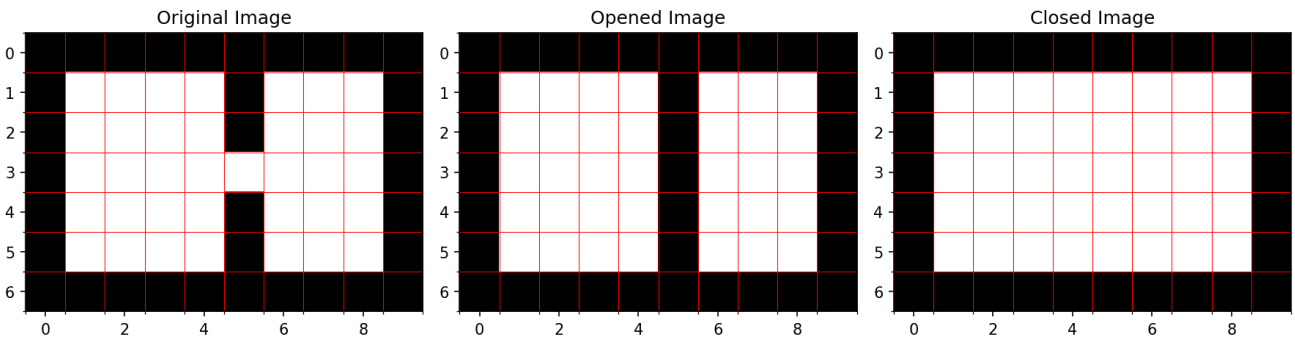

Figure 4: Opening and Closing. Left: Original shape with a bridge. Center: Opened image (bridge broken). Right: Closed image (bridge expanded and holes filled).

4.4 Discussion

The sequence of operations dictated the morphological outcome. In the Opening operation, the initial erosion completely obliterated the narrow bridge because the structuring element could not fit inside it. The subsequent dilation restored the primary blocks to their original size, but the bridge remained permanently erased. In the Closing operation, the initial dilation expanded the shapes, merging them across the gap. The subsequent erosion brought the outer boundaries back to normal, but the merged gap remained filled.

4.5 Conclusion

The utility of compound morphological operations was successfully demonstrated. It was verified that Opening is an effective tool for separating loosely connected objects and removing noise, while Closing is highly effective at fusing breaks and filling internal voids within objects.

5 Experiment 9: Hit-or-Miss Transform

5.1 Theory and Introduction

While the Hit-or-Miss Transform is useful for strict, pixel-perfect binary shapes, real-world scanned images often require more robust pattern recognition techniques. **Template Matching** is utilized to find areas of an image that are similar to a smaller template patch [1].

To measure this similarity, **Normalized Cross-Correlation (NCC)** is commonly employed. NCC slides the template across the main image and calculates a correlation coefficient at each position. The normalized equation ensures that the detection is robust against global lighting or contrast variations, returning a value between -1 (perfect mismatch) and 1 (perfect match).

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') - \bar{T})(I(x+x',y+y') - \bar{I})}{\sqrt{\sum_{x',y'}(T(x',y') - \bar{T})^2 \sum_{x',y'}(I(x+x',y+y') - \bar{I})^2}}$$

where T is the template, I is the image, and the overbars denote the mean values of the respective regions.

5.2 Methodology

A real-world scanned text image was processed to detect specific alphabetical characters. A Region of Interest (ROI) containing the target letter was interactively selected and used as the mathematical template to search the broader image.

5.2.1 Image Preparation and ROI Selection

The input image was converted to grayscale and binarized using Otsu's thresholding method to ensure consistent matching. An interactive UI was triggered, allowing the user to draw a bounding box around the letter "e" to define the template matrix.

5.2.2 Python Implementation

The OpenCV library function `cv2.matchTemplate` was utilized with the `TM_CCOEFF_NORMED` correlation method. To prevent a single letter from generating multiple overlapping detections, Non-Maximum Suppression (NMS) was applied using `peak_local_max`. This isolated only the highest correlation peaks that exceeded a threshold of 0.8 and enforced a minimum distance between detections.

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import peak_local_max

# Step 1: Read the image
img = cv2.imread("Letters.jpeg")
if img is None:
    raise FileNotFoundError("Could not read 'Letters.jpeg'. Please check the path.")

# Convert BGR to RGB for correct matplotlib display later
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Step 2: Pre-processing
if len(img.shape) == 3:
    img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
else:
    img_gray = img

# Binarize the image (using Otsu's thresholding for robust automatic calculation)
_, bw = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

# Step 3: Select the pattern (Interactive)
print("Please select the ROI in the popped up window and press ENTER or SPACE.")
roi = cv2.selectROI(
```

```python
    "Select Pattern (Press ENTER when done)", bw, showCrosshair=True, fromCenter=False
)
cv2.destroyWindow("Select Pattern (Press ENTER when done)")

# Extract coordinates and crop the pattern
x, y, w, h = roi
if w == 0 or h == 0:
    raise ValueError("No ROI selected. Exiting.")
pattern = bw[y : y + h, x : x + w]

# Step 4: Perform Template Matching
corr_map = cv2.matchTemplate(bw, pattern, cv2.TM_CCOEFF_NORMED)

# Step 5: Find peaks and apply Non-Maximum Suppression
threshold = 0.8

coordinates = peak_local_max(
    corr_map, min_distance=min(w, h) // 2, threshold_abs=threshold
)

# Step 6: Display Results
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.canvas.manager.set_window_title("Pattern Detection Results")

# Subplot 1: The Template
axes[0].imshow(pattern, cmap="gray")
axes[0].set_title(f"Pattern to find (Size: {h}x{w})")
axes[0].axis("off")

# Subplot 2: Original Image with Detections
axes[1].imshow(img_rgb)
axes[1].set_title(f"Detected Locations: {len(coordinates)}")
axes[1].axis("off")

# Draw rectangles and center marks around every match found
for pt in coordinates:
    y_peak, x_peak = pt

    # Draw Rectangle
    rect = plt.Rectangle(
        (x_peak, y_peak), w, h, edgecolor="red", facecolor="none", linewidth=2
    )
    axes[1].add_patch(rect)

    # Draw Center Cross (+)
    center_x = x_peak + w / 2
    center_y = y_peak + h / 2
    axes[1].plot(center_x, center_y, "y+", markersize=10, markeredgewidth=2)

plt.tight_layout()
plt.show()

print(f"Finished! Found {len(coordinates)} instances of the pattern.")
```

5.3 Results

The template matching algorithm successfully scanned the document and located the requested pattern.

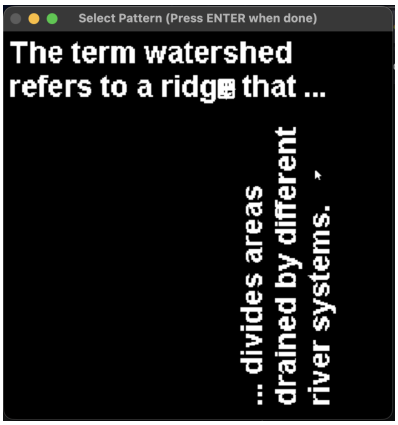

Figure 5: Interactive ROI Selection. The user defines the target template by drawing a bounding box around the specific pattern (the letter "e") directly on the binarized input image.

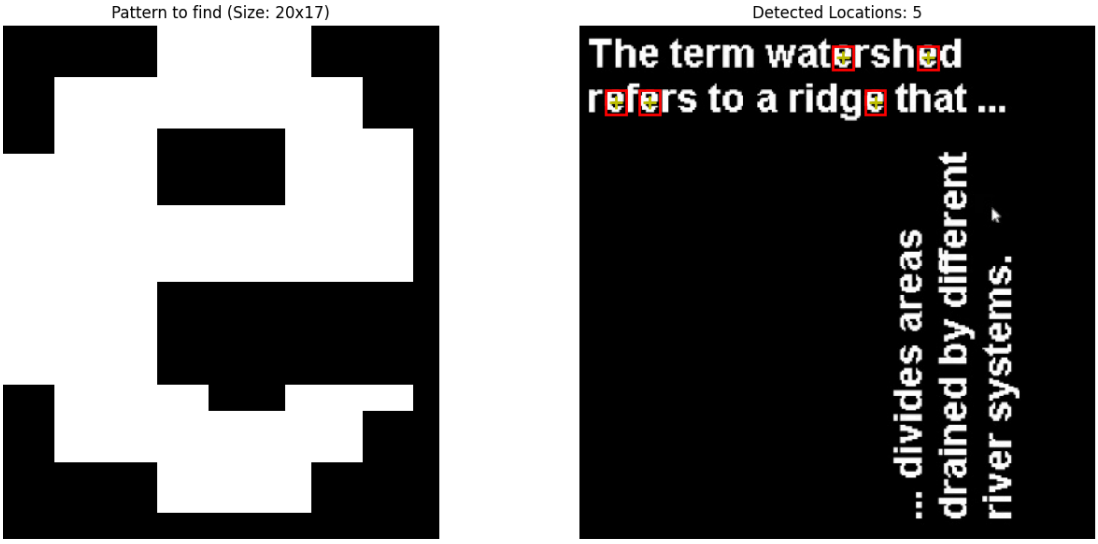

Figure 6: Template Matching Results. Left: The extracted 20×17 binary template of the letter "e". Right: The detection results, showing 5 localized instances marked with red bounding boxes and yellow center coordinates.

5.4 Discussion

The Normalized Cross-Correlation approach proved highly effective for feature detection in a noisy, real-world scan. As seen in the results, all 5 instances of the lowercase "e" in the horizontal text were perfectly isolated. The algorithm successfully ignored similar geometric shapes, such as the rounded bowls in the letters "d" and "a", because their cross-correlation coefficients fell below the 0.8 threshold. Furthermore, the application of Non-Maximum Suppression ensured that exactly one bounding box was drawn per letter, effectively filtering out redundant local peaks.

5.5 Conclusion

The robustness of template matching via Normalized Cross-Correlation was validated for optical character recognition tasks. It was demonstrated that by combining correlation mapping with peak-finding and non-maximum suppression, highly specific, complex morphological patterns can be accurately localized within a broader image matrix without the strict pixel-perfect constraints of traditional morphological transforms.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Boston, MA, USA: Pearson, 2018.